

```
#!/usr/bin/env python3
"""
SEALENS Dashboard – Hourly Refresh Script
=====
Fetches live data from free public APIs and patches index.html in place.
Designed to run in GitHub Actions on a cron schedule.

Data sources (all free, no API keys required):
- Oil prices      : FRED (St. Louis Fed / EIA) – CSV endpoint, CORS-open
- Chokepoints     : IMF PortWatch ArcGIS REST API – public JSON
- News           : Maritime Executive RSS feed via feedparser

Run locally:
    pip install requests feedparser
    python refresh.py

GitHub Actions runs this automatically every hour via .github/workflows/refresh.y
"""

import re
import sys
import json
import math
import logging
import requests
import feedparser
from datetime import datetime, timezone, timedelta
from pathlib import Path

# _____
# CONFIG
# _____

DASHBOARD_FILE = Path("index.html")
LOG_FILE        = Path("refresh.log")

# FRED series IDs (EIA daily prices, genuinely CORS-open)
FRED_BASE      = "https://fred.stlouisfed.org/graph/fredgraph.csv"
FRED_BRENT     = "DCOILBRENTU"    # Brent crude, $/bbl
FRED_WTI       = "DCOILWTICO"     # WTI crude, $/bbl

# IMF PortWatch ArcGIS REST endpoint
PORTWATCH_BASE = (
    "https://services9.arcgis.com/weJlQsnbMYJlCHdG/arcgis/rest/"
```

```

    "services/Daily_Chokepoints_Data/FeatureServer/0/query"
)
# Chokepoint IDs (from IMF PortWatch data explorer)
PORTWATCH_IDS = {
    "hormuz" : ("CHOKEPOINT7", ["CHOKEPOINT6", "CHOKEPOINT8"]),
    "bab"    : ("CHOKEPOINT1", ["CHOKEPOINT2", "CHOKEPOINT3"]),
    "panama" : ("CHOKEPOINT12", ["CHOKEPOINT11", "CHOKEPOINT13"]),
}
# Pre-crisis baselines (vessels/day, all-vessel types per IMF PortWatch)
BASELINES = {"hormuz": 96, "bab": 56, "panama": 37}

# RSS feeds (tried in order, first success wins)
RSS_FEEDS = [
    "https://maritime-executive.com/shipping-news/rss",
    "https://gcaptain.com/feed/",
    "https://splash247.com/feed/",
]
NEWS_COUNT = 10    # how many news items to show

# SGT = UTC+8
SGT = timezone(timedelta(hours=8))

# Request timeout
TIMEOUT = 15

# -----
#  LOGGING
# -----

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s  %(levelname)-7s  %(message)s",
    handlers=[
        logging.StreamHandler(sys.stdout),
        logging.FileHandler(LOG_FILE, encoding="utf-8"),
    ],
)
log = logging.getLogger("sealens")

# -----
#  HELPERS
# -----

def now_sgt() -> datetime:
    return datetime.now(SGT)

```

```

def fmt_date_long(dt: datetime) -> str:
    """Wednesday, 13 May 2026"""
    return dt.strftime("%-d %B %Y").lstrip()

def fmt_date_short(dt: datetime) -> str:
    """13 May"""
    return dt.strftime("%-d %b").lstrip()

def fmt_time(dt: datetime) -> str:
    """07:42"""
    return dt.strftime("%H:%M")

def week_number(dt: datetime) -> int:
    return dt.isocalendar()[1]

def quarter(dt: datetime) -> int:
    return (dt.month - 1) // 3 + 1

def pct_change(new: float, old: float) -> float:
    if old == 0:
        return 0.0
    return (new - old) / old * 100

def arrow(value: float) -> str:
    return "▲" if value >= 0 else "▼"

def parse_fred_csv(text: str) -> list[tuple[str, float]]:
    """
    Parse FRED CSV (DATE,VALUE) and return list of (date_str, price) tuples,
    most-recent last, ignoring missing-value dots.
    """
    rows = []
    for line in text.strip().splitlines():
        line = line.strip()
        if not line or line.upper().startswith("DATE"):
            continue
        parts = line.split(",")
        if len(parts) != 2:
            continue

```

```

        date_str, val_str = parts
        val_str = val_str.strip()
        if val_str == "." or val_str == "":
            continue
        try:
            rows.append((date_str.strip(), float(val_str)))
        except ValueError:
            continue
    return rows

# -----
# 1. OIL PRICES - FRED (St. Louis Fed / EIA)
# -----

def fetch_oil_price(series_id: str) -> dict:
    """
    Returns dict with keys:
        price, prev_price, date, prev_date, pct_chg, source
    Falls back to None values on failure.
    """
    url = f"{FRED_BASE}?id={series_id}"
    try:
        resp = requests.get(url, timeout=TIMEOUT)
        resp.raise_for_status()
        rows = parse_fred_csv(resp.text)
        if len(rows) < 2:
            raise ValueError(f"Only {len(rows)} data points returned")
        last_date, last_price = rows[-1]
        prev_date, prev_price = rows[-2]
        chg = pct_change(last_price, prev_price)
        log.info(f"FRED {series_id}: ${last_price:.2f} ({last_date}), "
                 f"$prev ${prev_price:.2f}, {chg:+.2f}%")
        return {
            "price"      : last_price,
            "prev_price" : prev_price,
            "date"       : last_date,
            "prev_date"  : prev_date,
            "pct_chg"    : chg,
            "source"     : "FRED/EIA",
            "ok"         : True,
        }
    except Exception as exc:
        log.warning(f"FRED {series_id} failed: {exc}")
        return {"ok": False}

```

```

def fetch_all_oil() -> dict:
    brent = fetch_oil_price(FRED_BRENT)
    wti    = fetch_oil_price(FRED_WTI)
    return {"brent": brent, "wti": wti}

# -----
# 2. CHOKEPOINT TRANSITS – IMF PortWatch ArcGIS
# -----

def fetch_portwatch(portid: str, limit: int = 14) -> list[dict]:
    """
    Fetch recent transit records for a single chokepoint ID.
    Returns list of attribute dicts, most-recent first.
    """
    params = {
        "where"           : f"portid='{portid}'",
        "outFields"       : "portid,portname,date,calls,volume",
        "orderByFields"   : "date DESC",
        "resultRecordCount" : limit,
        "f"               : "json",
    }
    resp = requests.get(PORTWATCH_BASE, params=params, timeout=TIMEOUT)
    resp.raise_for_status()
    data = resp.json()
    features = data.get("features", [])
    if not features:
        raise ValueError(f"No features for portid={portid}")
    return [f["attributes"] for f in features]

def calc_ma(records: list[dict], days: int = 7) -> float | None:
    vals = [float(r["calls"]) for r in records[:days]
             if r.get("calls") is not None and float(r["calls"]) >= 0]
    return round(sum(vals) / len(vals), 1) if vals else None

def fetch_chokepoint(name: str) -> dict:
    """
    Tries primary ID then alternates. Returns:
        today, ma7, pct_of_baseline, portname, date_label, ok
    """
    primary, alternates = PORTWATCH_IDS[name]
    ids_to_try = [primary] + alternates
    baseline = BASELINES[name]

    for portid in ids_to_try:

```

```

try:
    records = fetch_portwatch(portid)
    latest = records[0]
    today = int(float(latest.get("calls", 0)))
    ma7 = calc_ma(records, 7)
    pct = round((today / baseline) * 100) if baseline else None
    portname = latest.get("portname", name.capitalize())
    # Parse date: FRED-style epoch ms or string
    raw_date = latest.get("date")
    if isinstance(raw_date, (int, float)):
        dt = datetime.fromtimestamp(raw_date / 1000, tz=timezone.utc)
        date_label = dt.strftime("%-d %b")
    else:
        date_label = str(raw_date)[:10] if raw_date else "recent"

    log.info(f"PortWatch {name} ({portid}/{portname}): "
            f"today={today}, 7dMA={ma7}, pct={pct}%")
    return {
        "today" : today,
        "ma7" : ma7,
        "pct" : pct,
        "neg_pct" : round(100 - pct) if pct is not None else None,
        "portname" : portname,
        "date_label" : date_label,
        "portid" : portid,
        "ok" : True,
    }
except Exception as exc:
    log.warning(f"PortWatch {name} id={portid} failed: {exc}")
    continue

log.error(f"All PortWatch IDs failed for {name}")
return {"ok": False}

def fetch_all_chokepoints() -> dict:
    return {name: fetch_chokepoint(name) for name in ("hormuz", "bab", "panama")}

# -----
# 3. NEWS — RSS feeds
# -----

# Category inference from keywords in headline/description
CATEGORY_MAP = [
    ("cat-geo", ["hormuz", "iran", "houthi", "red sea", "geopolit",
                 "trump", "war", "military", "ceasefire", "strait",

```

```

        "attack", "security", "sanctions", "conflict"]],
("cat-rate", ["rate", "freight", "scfi", "wci", "fbx", "spot",
               "surcharge", "efs", "pss", "gri", "tariff"]],
("cat-ma", ["acqui", "merger", "deal", "bid", "takeover", "purchase",
            "zim", "hapag", "consolidat"]],
("cat-reg", ["imo", "eu ets", "cii", "decarboni", "emission",
             "regulation", "mepc", "fueleu", "compliance",
             "carbon", "methanol", "lng", "green fuel"]],
("cat-cap", ["order", "newbuild", "capacity", "fleet", "vessel",
             "teu", "ulcv", "charter", "delivery", "shipyard"]],
("cat-ops", ["port", "congestion", "delay", "operation", "service",
             "routing", "alliance", "schedule", "reliability"]],
]

```

```

CAT_LABELS = {
    "cat-geo" : "Geopolitics",
    "cat-rate" : "Rates",
    "cat-ma" : "M&A",
    "cat-reg" : "Regulatory",
    "cat-cap" : "Capacity",
    "cat-ops" : "Operations",
}

```

```

def infer_category(title: str, description: str = "") -> tuple[str, str]:
    text = (title + " " + description).lower()
    for cat_class, keywords in CATEGORY_MAP:
        if any(kw in text for kw in keywords):
            return cat_class, CAT_LABELS[cat_class]
    return "cat-ops", "Shipping"

```

```

def strip_html(text: str) -> str:
    """Remove HTML tags and collapse whitespace."""
    text = re.sub(r"<[^>]+>", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

```

```

def fetch_news() -> list[dict]:
    """
    Try each RSS feed in order. Return list of dicts with:
        title, link, date_str, description, cat_class, cat_label
    """
    for feed_url in RSS_FEEDS:
        try:
            log.info(f"Fetching RSS: {feed_url}")

```

```

feed = feedparser.parse(feed_url)
entries = feed.entries
if not entries:
    log.warning(f" RSS empty: {feed_url}")
    continue

items = []
for entry in entries[:NEWS_COUNT]:
    title = strip_html(entry.get("title", "")).strip()
    link = entry.get("link", "#")
    desc = strip_html(entry.get("summary", ""))[:150]
    # Date
    published = entry.get("published_parsed")
    if published:
        dt = datetime(*published[:6], tzinfo=timezone.utc)
        date_str = dt.strftime("%-d %b %Y")
    else:
        date_str = ""

    cat_class, cat_label = infer_category(title, desc)
    items.append({
        "title" : title,
        "link" : link,
        "date_str" : date_str,
        "desc" : desc,
        "cat_class" : cat_class,
        "cat_label" : cat_label,
    })

if items:
    log.info(f" Got {len(items)} news items from {feed_url}")
    return items

except Exception as exc:
    log.warning(f" RSS error {feed_url}: {exc}")
    continue

log.error("All RSS feeds failed")
return []

# -----
# 4. HTML PATCHING - regex-based, surgical replacements
# -----

def sub(pattern: str, replacement: str, html: str,
        flags: int = re.DOTALL) -> str:

```



```

        """Single-replacement regex substitution with logging."""
        new_html, n = re.subn(pattern, replacement, html, count=1, flags=flags)
        if n == 0:
            log.warning(f" Pattern NOT matched: {pattern[:80]!r}")
        return new_html

def patch_header(html: str, dt: datetime) -> str:
    """Update date, time and week/quarter in the header."""
    log.info("Patching header...")
    day_name = dt.strftime("%A")
    day_num = dt.day
    month = dt.strftime("%B")
    year = dt.year
    date_long = f"{day_name}, {day_num} {month} {year}"
    week = week_number(dt)
    q = quarter(dt)
    time_str = fmt_time(dt)
    build_ts = dt.strftime("%-d %b %Y %H:%M SGT")

    html = sub(
        r'(<div class="hdr-d1">)[^<]*(</div>)',
        rf'\g<1>{date_long}\g<2>',
        html
    )
    html = sub(
        r'(<div class="hdr-d2">)Week \d+ \cdot Q\d \d+ \cdot \d+:\d+ SGT(</div>)',
        rf'\g<1>Week {week} \cdot Q{q} {year} \cdot {time_str} SGT\g<2>',
        html
    )
    html = sub(
        r'<!-- BUILD:.*?-->',
        f'<!-- BUILD: v5.0 auto-refresh \cdot {build_ts} -->',
        html
    )
    return html

def patch_oil(html: str, oil: dict, dt: datetime) -> str:
    """
    Patch the oil price section:
    - Section header date
    - Brent card: price, delta, note, source timestamp
    - WTI card: price, delta, note, spread
    - VLSFO estimate: derived from Brent  $\times$  6.62
    - CEO briefing paragraph
    """

```

```

log.info("Patching oil prices...")
date_short = fmt_date_short(dt)
brent = oil.get("brent", {})
wti    = oil.get("wti",    {})

# Section header
html = sub(
    r'(Energy Prices – Crude Oil & Bunker · As of )[^<]*(</div>)',
    rf'\g<1>{date_short}, {dt.year}\g<2>',
    html
)

# — Brent —
if brent.get("ok"):
    bp      = brent["price"]
    bc      = brent["pct_chg"]
    bd      = brent["date"]
    bprev   = brent["prev_price"]
    b_arr   = arrow(bc)
    b_sign  = "+" if bc >= 0 else ""
    b_col   = "c-teal" if bc >= 0 else "c-red"
    b_abs   = bp - bprev
    b_abs_sign = "+" if b_abs >= 0 else ""

    # price
    html = sub(
        r'(<div class="oil-price c-amber">)\$[\d.]+(<span class="oil-unit"> /
        rf'\g<1>${bp:.2f}\g<2>',
        html
    )
    # source timestamp in type label
    html = sub(
        r'(Brent Crude \ (ICE\ ) <span[>]*>·) [^<]*(</span></div>)',
        rf'\g<1> {bd} close · FRED/EIA\g<2>',
        html
    )
    # delta
    html = sub(
        r'(<div class="oil-delta " [^"]*( "[^/"]*/div>)',
        rf'\g<1>{b_col}\ ">{b_arr} {b_sign}{bc:.2f}% · {b_abs_sign}${abs(b_abs
        html, flags=0
    )
else:
    log.warning("Brent data unavailable – keeping existing values")

# — WTI —
if wti.get("ok"):

```

```

wp      = wti["price"]
wc      = wti["pct_chg"]
wd      = wti["date"]
w_arr   = arrow(wc)
w_sign= "+" if wc >= 0 else ""
w_col   = "c-teal" if wc >= 0 else "c-red"
spread  = round(brent["price"] - wp, 2) if brent.get("ok") else "-"

html = sub(
  r'(<div class="oil-price c-gold">)\$[\d.]+(<span class="oil-unit"> /b
  rf'\g<1>${wp:.2f}\g<2>',
  html
)
html = sub(
  r'(WTI Crude \ (NYMEX\ ) <span[^>]*>·) [^<]*(</span></div>)',
  rf'\g<1> {wd} close · FRED/EIA\g<2>',
  html
)
html = sub(
  r'(<div class="oil-type">WTI.*?</div>\s*<div class="oil-price.*?</div
  rf'\g<1>{w_col}\ ">{w_arr} {w_sign}{wc:.2f}% vs prev close</div>',
  html
)
# spread note
html = sub(
  r'(Brent-WTI spread ~\$) [\d.] +',
  rf'\g<1>{spread}',
  html
)

# — VLSFO estimate (Brent × 6.62) —
if brent.get("ok"):
  vlsfo = round(brent["price"] * 6.62)
  rotterdam_est = round(vlsfo * 0.970) # Rotterdam ~3% cheaper historical
  html = sub(
    r'(<div class="oil-price c-teal">)\$[\d.]+(<span class="oil-unit"> /MT
    rf'\g<1>~${vlsfo}\g<2>',
    html
  )
  html = sub(
    r'(VLSFO Singapore <span[^>]*>·) [^<]*(</span></div>)',
    rf'\g<1> {date_short} est. · Ship&Bunker\g<2>',
    html
  )
  html = sub(
    r'(Rotterdam ~\$) [\d.] + ( \ (cheapest\ ) )',
    rf'\g<1>{rotterdam_est}\g<2>',

```

```

        html
    )

# — CEO briefing date label —
html = sub(
    r'(\⚡ CEO Energy Briefing — )[^<]*(</div>)',
    rf'\g<1>{date_short}, {dt.year}\g<2>',
    html
)

return html

def patch_chokepoints(html: str, choke: dict, dt: datetime) -> str:
    """
    Patch transit counts, percentages, date labels,
    status pills and source notes for all three chokepoints.
    """
    log.info("Patching chokepoints...")
    date_short = fmt_date_short(dt)

    # Section header
    html = sub(
        r'(Global Chokepoint Daily Transits — IMF PortWatch · Last Updated: )[^<]
        rf'\g<1>Tue {date_short}, {dt.year}\g<2>',
        html
    )

    # — HORMUZ —
    h = choke.get("hormuz", {})
    if h.get("ok"):
        today      = h["today"]
        ma7         = int(h["ma7"]) if h["ma7"] is not None else "-"
        neg_pct     = h["neg_pct"]
        day_n       = (dt.date() - datetime(2026, 2, 28, tzinfo=SGT).date()).days
        bar_pct     = max(1, today) # never zero-width bar

    # Status pill
    if today < 5:
        pill_txt = f"CLOSED — DAY {day_n}"
        pill_bg  = "var(--red)"; pill_fg = "#fff"
    elif today < 20:
        pill_txt = f"NEAR CLOSED — DAY {day_n}"
        pill_bg  = "var(--red)"; pill_fg = "#fff"
    elif today < 50:
        pill_txt = f"SEVERELY DISRUPTED — DAY {day_n}"
        pill_bg  = "var(--amber)"; pill_fg = "#000"

```

```

else:
    pill_txt = "RESTRICTED"
    pill_bg = "var(--amber)"; pill_fg = "#000"

html = sub(
    r'(<span class="choke-status-pill" style="background:var\(--(?:red|am
rf'\g<1>{pill_txt}\g<2>',
    html, flags=0
)
# Latest metric
html = sub(
    r'(<div class="choke-m-lbl">Yesterday \([^\)]+\)</div><div class="chok
rf'\g<1>{today}\g<2>',
    html
)
# 7d MA metric
html = sub(
    r'(<div class="choke-m-lbl">7-Day MA</div><div class="choke-m-val c-(
rf'\g<1>{ma7}\g<2>',
    html
)
# vs-pct bars
html = sub(
    r'(<div class="vs-fill" style="width:)\d+(\%";background:var\(--red\)"
rf'\g<1>{bar_pct}\g<2>-{neg_pct}%\g<3>',
    html
)
# Source note
html = sub(
    r'(\IMF PortWatch \(\last: Tue \)[^\)]+\)\)',
    rf'\g<1>{date_short}, {dt.year}\g<2>',
    html
)
# SVG annotation
html = sub(
    r'(<text[^\>]*>(?:Apr|May|Mar) \d+[^\<]*</text>)',
    rf'\g<1>{date_short}: {today}\g<2>',
    html, flags=0
)
log.info(f" Hormuz: {today} today, 7dMA {ma7}, -{neg_pct}%")

# — BAB EL-MANDEB —
b = choke.get("bab", {})
if b.get("ok"):
    b_today = b["today"]
    b_ma7 = int(b["ma7"]) if b["ma7"] is not None else "-"
    b_neg = b["neg_pct"]

```

```

baseline = BASELINES["bab"]
b_bar     = round((b_today / baseline) * 100)

html = sub(
    r'(<div class="choke-m-lbl">7d MA \ (wk [^])+\)</div><div class="choke
    rf'\g<1>{b_ma7}\g<2>{b_today}',
    html
)
html = sub(
    r'(<div class="vs-fill" style="width:\d+(\%);background:var\(--amber\
    rf'\g<1>{b_bar}\g<2>--{b_neg}\g<3>',
    html
)
log.info(f" Bab el-Mandeb: {b_today} today, 7dMA {b_ma7}")

# — PANAMA —
p = choke.get("panama", {})
if p.get("ok"):
    p_today = p["today"]
    p_ma7    = int(p["ma7"]) if p["ma7"] is not None else "-"
    baseline = BASELINES["panama"]
    p_pct    = round((p_today / baseline) * 100)

    html = sub(
        r'(<div class="choke-m-lbl">7d MA \ (wk [^])+\)</div><div class="choke
        rf'\g<1>{p_ma7}\g<2>{p_today}',
        html
    )
    log.info(f" Panama: {p_today} today, 7dMA {p_ma7}, {p_pct}% of baseline"

return html

def patch_signal_bar(html: str, oil: dict, choke: dict, dt: datetime) -> str:
    """
    Update the 8-card signal bar at the top:
    - Hormuz transit value, delta, note
    - Brent price, delta, note
    - VLSFO estimate
    - Date stamps in sig-delta / sig-note fields
    """
    log.info("Patching signal bar...")
    date_short = fmt_date_short(dt)

    # Hormuz
    h = choke.get("hormuz", {})
    if h.get("ok"):

```

```

today = h["today"]
ma7    = int(h["ma7"]) if h["ma7"] is not None else "-"
neg    = h["neg_pct"]
day_n  = (dt.date() - datetime(2026, 2, 28, tzinfo=SGT).date()).days
html = sub(
    r'(<div class="sig-lbl">Hormuz Transits</div>\s*<div class="sig-val">
    rf'\g<1>{today}\g<2>vessels {date_short} · 7d MA: ~{ma7}\g<3>vs 96/da
    html
)

# Brent
brent = oil.get("brent", {})
if brent.get("ok"):
    bp  = brent["price"]
    bc  = brent["pct_chg"]
    arr = arrow(bc)
    sgn = "+" if bc >= 0 else ""
    col = "c-teal" if bc >= 0 else "c-red"
    bd  = brent["date"]
    html = sub(
        r'(<div class="sig-lbl">Brent Crude</div>\s*<div class="sig-val">)[^<
        rf'\g<1>${bp:.2f}\g<2>{col}\>{arr} {sgn}{bc:.2f}% · {bd}\g<3>+{round
        html, flags=0
    )

# VLSFO estimate
if brent.get("ok"):
    vlsfo = round(brent["price"] * 6.62)
    rotterdam = round(vlsfo * 0.970)
    html = sub(
        r'(<div class="sig-lbl">VLSFO Singapore</div>\s*<div class="sig-val">
        rf'\g<1>~${vlsfo}\g<2>',
        html
    )
    html = sub(
        r'(sig-note">Est\. from Brent · Rotterdam ~\${\d}+(</div>)',
        rf'\g<1>{rotterdam}\g<2>',
        html
    )

return html

```

```
def patch_news(html: str, news_items: list[dict]) -> str:
```

```
"""
```

Replace the entire news list block between the card-hdr and the closing </div of the news card. Uses a reliable sentinel pattern.

```

"""
if not news_items:
    log.warning("No news items to patch - keeping existing")
    return html

log.info(f"Patching news ({len(news_items)} items)...")

# Build new news HTML
lines = []
for i, item in enumerate(news_items[:NEWS_COUNT], 1):
    title = item["title"].replace("&", "&amp;").replace("<", "&lt;").repla
    link = item["link"]
    date_str = item["date_str"]
    desc = item["desc"].replace("&", "&amp;").replace("<", "&lt;").replac
    cat_cls = item["cat_class"]
    cat_lbl = item["cat_label"]
    num = str(i).zfill(2)
    desc_html = f'\n        <div class="news-sw">↳ {desc}...</div>' if desc else

    lines.append(f"""
<div class="news-item"><div class="news-num">{num}</div><div class="news-body
    <span class="news-cat {cat_cls}">{cat_lbl}</span>
    <div class="news-hl"><a href="{link}" target="_blank" rel="noopener">{title
    <div class="news-meta">{date_str} · Live RSS</div>{desc_html}
</div></div>""")

new_news_block = "\n".join(lines) + "\n "

# Replace everything between the card-hdr closing tag and the end of the news
html = sub(
    r'(Top 10 News – Container Shipping Intelligence</div><span[^>]*>[^<]*</s
    r'(?<div class="news-item">.*?</div>\s*)+',
    rf'\g<1>\n{new_news_block}',
    html
)

return html

def patch_date_stamp(html: str, section_tag: str, dt: datetime) -> str:
    """Generic helper: update a tag-type date label anywhere in the HTML."""
    date_short = fmt_date_short(dt)
    year = dt.year
    html = sub(
        rf'({re.escape(section_tag)} · ) [A-Z] [a-z]+ \d{{1,2}}, \d{{4}}',
        rf'\g<1>{date_short}, {year}',
        html

```



```

)
return html

def patch_all_section_dates(html: str, dt: datetime) -> str:
    """Update every static date label in section headers and tags."""
    log.info("Patching section date labels...")
    date_short = fmt_date_short(dt)
    year = dt.year

    # Energy section header
    html = sub(
        r'(Energy Prices – Crude Oil & Bunker · As of )[^<]*(</div>)',
        rf'\g<1>{date_short}, {year}\g<2>',
        html
    )

    # Bunkering section header
    html = sub(
        r'(Global Bunkering Prices – 13 Major Ports · Ship&Bunker · )[^<]*(</div>)',
        rf'\g<1>{date_short}, {year}\g<2>',
        html
    )

    # Bunkering tag
    html = sub(
        r'(<span class="tag tg-watch">Est\.[A-Z][a-z]+ \d{{1,2}}, \d{{4}})',
        rf'\g<1>{date_short} · Ship&Bunker\g<2>',
        html
    )

    # Region headers inside bunkering table
    for region in ["ASIA-PACIFIC", "MIDDLE EAST", "EUROPE", "AMERICAS", "CAPE ROU
        html = sub(
            rf'(>{region} · Prices est\.[A-Z][a-z]+ \d{{1,2}}, \d{{4}})',
            rf'\g<1>{date_short}, {year}',
            html
        )

    # EUA date
    html = sub(
        r'(EUA Price · [A-Z][a-z]+ \d{1,2})',
        rf'\g<1>{date_short}',
        html
    )

    # Green fuels date
    html = sub(
        r'(Green & Alternative Fuels – Availability & Price · )[^<]*(</div>)',
        rf'\g<1>{date_short}, {year} (est.)\g<2>',
        html
    )

```

```

# News card tag
html = sub(
    r'(<span class="tag tg-live">)\d{1,2} [A-Z][a-z]+ \d{4}(</span>)',
    rf'\g<1>{date_short} {year}\g<2>',
    html
)

# Rates section header
html = sub(
    r'(Market & Freight Rates · SCFI [\d,]+ · WCI \${[\d,]+ · })[A-Z][a-z]+',
    rf'\g<1>{date_short}, {year}\g<2>',
    html
)

# Callout dates
for callout in ["Cheapest VLSFO", "Fujairah", "Asian Hub Availability"]:
    html = sub(
        rf'(✓ {callout} · |🔴 {callout} · |⚠ {callout} · ) [A-Z][a-z]+ \d{{1,2}}',
        rf'\g<1>{date_short}',
        html
    )
return html

# -----
# 5. MAIN
# -----

def main() -> int:
    dt = now_sgt()
    log.info("=" * 60)
    log.info(f"SEALENS refresh starting - {dt.strftime('%A, %-d %B %Y %H:%M SGT')}")
    log.info("=" * 60)

    # — Load dashboard —
    if not DASHBOARD_FILE.exists():
        log.error(f"Dashboard file not found: {DASHBOARD_FILE}")
        return 1

    html = DASHBOARD_FILE.read_text(encoding="utf-8")
    log.info(f"Loaded {DASHBOARD_FILE} ({len(html):,} bytes)")

    # — Fetch all data —
    oil    = fetch_all_oil()
    choke  = fetch_all_chokepoints()
    news   = fetch_news()

    # — Count successes —
    oil_ok = sum(1 for v in oil.values() if v.get("ok"))

```

```

choke_ok = sum(1 for v in choke.values() if v.get("ok"))
news_ok = len(news)
log.info(f>Data fetched - oil:{oil_ok}/2  chokepoints:{choke_ok}/3  news:{new

# — Patch HTML —
html = patch_header(html, dt)
html = patch_oil(html, oil, dt)
html = patch_chokepoints(html, choke, dt)
html = patch_signal_bar(html, oil, choke, dt)
html = patch_news(html, news)
html = patch_all_section_dates(html, dt)

# — Write back —
DASHBOARD_FILE.write_text(html, encoding="utf-8")
log.info(f"Dashboard updated - {len(html):,} bytes written to {DASHBOARD_FILE
log.info(f"Refresh complete at {now_sgt().strftime('%H:%M:%S SGT')}")
log.info("=" * 60)

# Return non-zero only if ALL fetches failed (don't fail the Action for parti
if oil_ok == 0 and choke_ok == 0 and news_ok == 0:
    log.error("All data fetches failed - dashboard not meaningfully updated")
    return 1
return 0

if __name__ == "__main__":
    sys.exit(main())

```